

FACULDADE DE CIÊNCIAS APLICADAS E SOCIAIS DE PETROLINA
CURSO DE CIÊNCIA DA COMPUTAÇÃO
DISCIPLINA DE MACHINE LEARNING — AV2

**CLASSIFICAÇÃO DE TRÁFEGO DE REDE COM MACHINE
LEARNING:**
UM ESTUDO COMPARATIVO SOBRE O DATASET NSL-KDD

Trabalho apresentado por:

Edmilson Breno R. Luna

Francklin Leandro R. I. Bartilotti

Docente: Prof. Mateus Silva

RESUMO

Este trabalho apresenta um estudo comparativo de três algoritmos de Machine Learning — Árvore de Decisão, Random Forest e K-Nearest Neighbors (KNN) — aplicados à classificação de tráfego de rede como Normal ou Ataque, utilizando o dataset NSL-KDD. O pipeline cobre o ciclo completo de um projeto de aprendizado de máquina: análise exploratória de dados (EDA), pré-processamento com controle de vazamento de dados, treinamento com hiperparâmetros fixos e reproduzíveis, validação cruzada estratificada 5-fold com OOB Score, avaliação por F1-Score e AUC-ROC, e comparação estatística via Teste de McNemar. Os resultados indicam que o Random Forest obteve o melhor desempenho geral (F1-Score = 0,7900; AUC-ROC = 0,8541; OOB Score = 0,9998), com diferença estatisticamente significativa em relação ao KNN ($p < 0,05$). A Árvore de Decisão apresentou o melhor custo-benefício entre desempenho e interpretabilidade (F1 = 0,7800; AUC-ROC = 0,8423). As principais limitações incluem o uso de um dataset gerado em 1999 e o Recall moderado ($\sim 65\%$), crítico em aplicações reais de segurança de redes.

Palavras-chave: detecção de intrusão; NSL-KDD; classificação binária; Random Forest; Machine Learning; validação estatística.

Sumário

1	INTRODUÇÃO	3
2	OBJETIVOS	3
2.1	Objetivo Geral	3
2.2	Objetivos Específicos	4
3	FUNDAMENTAÇÃO TEÓRICA	4
3.1	Deteccção de Intrusão em Redes e o Dataset NSL-KDD	4
3.2	Árvores de Decisão	4
3.3	Random Forest e Bagging	5
3.4	K-Nearest Neighbors (KNN)	5
3.5	Métricas de Avaliação	6
3.6	Teste de McNemar	6
3.7	Prevenção de Vazamento de Dados	6
4	METODOLOGIA	6
4.1	Dataset e Variável-Alvo	7
4.2	Fontes	7
4.3	Procedimentos Metodológicos	7
4.4	Limitações Metodológicas	8
5	RESULTADOS	8
5.1	Análise Exploratória de Dados	8
5.2	Desempenho dos Modelos	8
5.3	Validação Cruzada e OOB Score	9
5.4	Teste de McNemar	9
5.5	Importância de Features	9
6	DISCUSSÃO	10
7	LIMITAÇÕES E AMEAÇAS À VALIDADE	11
7.1	Validade Interna	11
7.2	Validade Externa	11
7.3	Ameaças ao Desempenho	11
8	CRONOGRAMA	11
9	CONCLUSÃO	12
	Referências Bibliográficas	12

1. INTRODUÇÃO

A segurança da informação tornou-se um dos pilares fundamentais da infraestrutura digital moderna. Com o crescimento exponencial dos ataques cibernéticos — que evoluíram de simples invasões a operações coordenadas capazes de paralisar infraestruturas críticas — a detecção automática de tráfego malicioso passou a ser considerada uma necessidade estratégica para organizações públicas e privadas [1].

Sistemas de Detecção de Intrusão (IDS) baseados em Machine Learning apresentam uma vantagem fundamental em relação às abordagens tradicionais baseadas em assinaturas: a capacidade de identificar padrões anômalos sem depender de regras predefinidas, adaptando-se dinamicamente a novos vetores de ataque. Essa flexibilidade é especialmente relevante em um cenário onde surgem diariamente novas variantes de ataques e técnicas de evasão [2].

O dataset NSL-KDD, derivado do KDD Cup 1999, é o benchmark mais utilizado na literatura para avaliação de algoritmos de classificação de tráfego de rede. Em relação ao KDD original, o NSL-KDD elimina registros duplicados, corrige desequilíbrios artificiais entre classes e fornece um conjunto de teste propositalmente mais desafiador — o NSL-KDD Test+ —, tornando os resultados mais realistas e comparáveis com a literatura científica [2].

Neste contexto, o presente trabalho propõe o desenvolvimento de um pipeline completo de Machine Learning para classificação binária de tráfego de rede (Normal vs. Ataque), comparando três paradigmas de aprendizado de máquina: a Árvore de Decisão (modelo interpretável e auditável), o Random Forest (ensemble baseado em bagging) e o KNN (modelo baseado em distância, sem fase de treinamento explícita). O estudo cobre todas as etapas do ciclo de um projeto de ML: análise exploratória, pré-processamento, treinamento, validação e interpretação dos resultados.

A motivação para este trabalho reside na necessidade de identificar modelos que equilibrem desempenho preditivo, interpretabilidade e custo computacional, em um domínio onde os erros têm impacto direto na segurança de sistemas e dados organizacionais.

Este trabalho tem como objetivos: (1) treinar e comparar três modelos de classificação com configurações reproduzíveis; (2) avaliar os desempenhos com métricas adequadas ao desbalanceamento de classes; (3) validar estatisticamente as diferenças entre modelos; e (4) identificar as features mais relevantes para a discriminação entre tráfego normal e malicioso.

2. OBJETIVOS

2.1 Objetivo Geral

Desenvolver e avaliar um pipeline completo de Machine Learning para classificação de tráfego de rede como Normal ou Ataque, utilizando o dataset NSL-KDD, e comparar o desempenho de três algoritmos distintos por meio de métricas estatisticamente validadas.

2.2 Objetivos Específicos

- Descrever e aplicar as etapas de análise exploratória de dados (EDA) no dataset NSL-KDD, identificando padrões relevantes para a discriminação entre tráfego normal e malicioso.
- Implementar o pré-processamento dos dados com controle rigoroso de vazamento de dados (*data leakage*), garantindo que as transformações sejam aprendidas exclusivamente no conjunto de treino.
- Treinar e comparar três modelos de Machine Learning — Árvore de Decisão, Random Forest e KNN — com hiperparâmetros fixos e reproduzíveis, rastreados em arquivo de experimentos.
- Avaliar os modelos por meio de métricas adequadas ao contexto de classes parcialmente desbalanceadas: F1-Score, AUC-ROC, Precision e Recall.
- Validar estatisticamente a diferença de desempenho entre os dois melhores modelos por meio do Teste de McNemar.
- Interpretar a importância das features identificadas pelo Random Forest e relacioná-las aos padrões observados na análise exploratória.
- Discutir as limitações do pipeline, as ameaças à validade dos resultados e as direções para trabalhos futuros.

3. FUNDAMENTAÇÃO TEÓRICA

3.1 Detecção de Intrusão em Redes e o Dataset NSL-KDD

A detecção de intrusão em redes é uma das áreas mais ativas na interseção entre segurança da informação e aprendizado de máquina. Um IDS tem como função monitorar o tráfego de uma rede e identificar comportamentos suspeitos ou maliciosos, gerando alertas para os administradores de segurança [1].

O dataset NSL-KDD foi proposto por Tavallaee et al. [1] como uma versão melhorada do KDD Cup 1999, eliminando os dois principais problemas do dataset original: a presença de registros duplicados (que causavam viés nos resultados) e a grande disparidade numérica entre classes de ataque, que mascarava a real capacidade discriminativa dos modelos. O NSL-KDD contém 125.973 registros de treino e 22.544 de teste, com 41 features distribuídas em quatro grupos: features básicas de conexão TCP/IP, features de conteúdo, features de tráfego em janela temporal de 2 segundos, e features baseadas no host de destino [2].

3.2 Árvores de Decisão

As Árvores de Decisão são modelos de aprendizado supervisionado que particionam recursivamente o espaço de atributos em regiões cada vez mais homogêneas, construindo uma

estrutura hierárquica de regras de decisão interpretáveis por humanos [7]. Em cada nó, o algoritmo seleciona o atributo e o limiar de divisão que maximizam a redução de impureza — medida pelo índice de Gini ou pela Entropia de Shannon.

O índice de Gini, adotado neste trabalho, é calculado como:

$$G = 1 - \sum_{k=1}^K p_k^2 \quad (1)$$

onde p_k é a proporção da classe k no nó. O Gini é computacionalmente mais eficiente que a Entropia por não envolver cálculo de logaritmos, produzindo resultados equivalentes na maioria dos problemas práticos.

A principal vantagem das Árvores de Decisão é a interpretabilidade: cada caminho da raiz até uma folha corresponde a uma regra de decisão auditável, o que facilita a explicação de alertas em ambientes regulados. A principal limitação é a alta variância — pequenas perturbações nos dados de treino podem produzir árvores significativamente diferentes [6].

3.3 Random Forest e Bagging

O Random Forest é um método ensemble que combina múltiplas Árvores de Decisão treinadas em amostras bootstrap do conjunto de treino — técnica conhecida como Bootstrap Aggregating (Bagging) [6]. A predição final é obtida por votação majoritária (classificação) ou média (regressão) das predições individuais.

A inovação central do Random Forest em relação ao Bagging simples é a aleatoriedade adicional na seleção de features: em cada nó de cada árvore, apenas $m \approx \sqrt{p}$ atributos são considerados para a divisão, onde p é o número total de features. Isso reduz a correlação entre as árvores e amplifica o efeito de redução de variância [6].

O OOB Score (*Out-of-Bag Score*) é uma estimativa não enviesada de generalização gerada gratuitamente durante o treinamento: em cada bootstrap, aproximadamente 36,8 % das amostras ficam fora da amostra de treino daquela árvore (out-of-bag) e são usadas para validação. O OOB Score funciona como uma validação cruzada automática, sem necessidade de conjunto de validação separado.

3.4 K-Nearest Neighbors (KNN)

O KNN é um algoritmo não paramétrico de aprendizado baseado em instâncias: para classificar uma nova amostra, identifica as k amostras mais próximas no espaço de features (por distância euclidiana) e atribui a classe majoritária entre elas [7]. O KNN não possui fase de treinamento explícita — simplesmente armazena os dados de treino —, mas apresenta alta latência na inferência, pois precisa calcular distâncias para todas as amostras do conjunto de treino a cada predição.

Por ser sensível à escala das features, o KNN exige normalização prévia dos dados. Neste trabalho, adotamos o `StandardScaler` para garantir que features como `src_bytes` (escala de

10⁹) não dominem o cálculo de distância em relação a indicadores binários como 1and.

3.5 Métricas de Avaliação

Em problemas de classificação com classes desbalanceadas e custos assimétricos de erro — como a detecção de intrusão —, a acurácia simples é uma métrica enganosa [4]. As métricas adotadas neste trabalho são:

F1-Score: média harmônica de Precision e Recall, que penaliza de forma equilibrada tanto alarmes falsos (FP) quanto ataques não detectados (FN):

$$F1 = 2 \cdot \frac{\text{Precision} \times \text{Recall}}{\text{Precision} + \text{Recall}} \quad (2)$$

AUC-ROC: Área sob a Curva ROC, que avalia a capacidade discriminativa do classificador independentemente do threshold de decisão. Com $AUC = 0,8541$, o Random Forest tem 85,4 % de probabilidade de atribuir score maior a uma conexão de ataque do que a uma conexão normal escolhida aleatoriamente.

3.6 Teste de McNemar

O Teste de McNemar é um teste estatístico não paramétrico aplicado para comparar dois classificadores binários avaliados no mesmo conjunto de teste [5]. Ao contrário de testes que comparam médias de métricas, o McNemar analisa diretamente a tabela de contingência das predições, levando em conta a correlação entre os modelos.

A hipótese nula (H_0) é de que os dois modelos erram nas mesmas amostras com igual frequência. Um $p < 0,05$ indica que as diferenças entre os modelos são estatisticamente significativas e não se devem ao acaso.

3.7 Prevenção de Vazamento de Dados

Vazamento de dados (*data leakage*) ocorre quando informações do conjunto de teste influenciam o pré-processamento ou o treinamento do modelo, inflacionando artificialmente as métricas de desempenho [3]. Neste trabalho, adotamos duas medidas para prevenir o vazamento: (1) a divisão treino/teste foi realizada antes de qualquer transformação dependente dos dados; e (2) o StandardScaler foi ajustado (*fit*) exclusivamente no conjunto de treino e aplicado (*transform*) ao conjunto de teste sem refitting.

4. METODOLOGIA

A pesquisa foi conduzida como um estudo experimental, com abordagem quantitativa e ênfase comparativa. O pipeline foi implementado em Python 3.13 com scikit-learn 1.8.0, pandas 2.2.2, numpy 1.26.4, matplotlib 3.10.0, seaborn 0.13.2 e statsmodels 0.14.2, com versões fixadas em `requirements.txt` para garantir reprodutibilidade.

4.1 Dataset e Variável-Alvo

O dataset NSL-KDD foi obtido do repositório público [2]:

- **Treino:** 125.973 amostras $\times$ 41 features
- **Teste:** 22.544 amostras $\times$ 41 features
- **Features:** 38 numéricas + 3 categóricas (protocol_type, service, flag)
- **Classes:** Normal (53,5 %) e Ataque (46,5 %) no treino

A variável-alvo original (23 tipos de ataque + normal) foi convertida em uma variável binária: Normal = 0 e Ataque = 1. Essa conversão é justificada pelo objetivo do projeto — detecção de intrusão (Normal vs. Ataque) —, não classificação multi-classe dos tipos de ataque.

4.2 Fontes

Foram utilizadas como fontes: artigos científicos indexados nas bases IEEE Xplore, ACM Digital Library e Google Scholar; documentação oficial do scikit-learn [7]; e o repositório público do NSL-KDD (https://github.com/defcom17/NSL_KDD).

4.3 Procedimentos Metodológicos

Etapa 1 — Análise Exploratória de Dados (EDA): Geração de cinco visualizações para identificar padrões relevantes antes da modelagem: distribuição das classes, tipo de protocolo por classe, histograma de src_bytes, heatmap de correlação das top 10 features e boxplot de dst_bytes por classe.

Etapa 2 — Pré-processamento: (a) Remoção da coluna difficulty (avaliação dos criadores do dataset, não uma feature real de tráfego — mantê-la seria data leakage); (b) codificação das 3 features categóricas com LabelEncoder — preferido ao OneHotEncoder para não expandir a dimensionalidade de 41 para 80+ features, dado que árvores de decisão são invariantes à ordem ordinal implícita; (c) normalização via StandardScaler ajustado exclusivamente no treino.

Etapa 3 — Treinamento: Três modelos foram instanciados com hiperparâmetros fixos: Árvore de Decisão (criterion=gini, random_state=42); Random Forest (n_estimators=100, oob_score=True, random_state=42); e KNN (n_neighbors=5, metric=minkowski). Todos os hiperparâmetros e resultados foram rastreados em experiments/experimentos.csv.

Etapa 4 — Validação: Validação cruzada estratificada 5-fold (F1-Score) no conjunto de treino; OOB Score do Random Forest; e avaliação final no conjunto de teste com acurácia, Precision, Recall, F1-Score e AUC-ROC.

Etapa 5 — Comparação Estatística: Teste de McNemar entre Árvore de Decisão e Random Forest ($\alpha = 0,05$).

Etapa 6 — Interpretação: Análise de importância de features via Mean Decrease in Impurity (MDI) do Random Forest e análise de erros (matriz de confusão, FP e FN).

4.4 Limitações Metodológicas

Os hiperparâmetros não foram otimizados via busca sistemática (GridSearchCV), o que pode subestimar o desempenho real dos modelos. O estudo está restrito ao dataset NSL-KDD, gerado em 1999, não representando ataques modernos. Os resultados não foram validados em capturas de tráfego real (arquivos PCAP de ambientes de produção).

5. RESULTADOS

5.1 Análise Exploratória de Dados

A EDA revelou cinco padrões com implicações diretas na modelagem:

Distribuição das Classes: 53,5 % Normal vs. 46,5 % Ataque — desbalanceamento moderado que tornaria a acurácia uma métrica enganosa, justificando o uso de F1-Score e AUC-ROC.

Protocolo por Classe: O protocolo icmp é dominado por ataques (>95 % dos registros), indicando alta relevância discriminativa. Conexões tcp e udp exigem análise de features adicionais.

Distribuição de src_bytes: Ataques concentram-se em dois padrões opostos — valores próximos de zero (varreduras e probes) e valores muito elevados (floods e DoS) — enquanto tráfego normal apresenta distribuição unimodal mais compacta.

Correlação entre Top 10 Features: As features src_bytes, dst_bytes, flag, logged_in e count são as mais correlacionadas com a variável-alvo — resultado confirmado pela importância de features do Random Forest, demonstrando consistência do pipeline.

Distribuição de dst_bytes: Conexões normais recebem mais bytes do destino, sugerindo que ataques frequentemente estabelecem conexões sem retorno de dados substancial (varreduras, tentativas mal-sucedidas).

5.2 Desempenho dos Modelos

A Tabela 1 apresenta os resultados comparativos no conjunto de teste.

Tabela 1: Resultados comparativos dos modelos no conjunto de teste

Modelo	Acurácia	Precision	Recall	F1	AUC-ROC
Árvore de Decisão	0,8200	0,9700	0,6500	0,7800	0,8423
Random Forest	0,8300	0,9700	0,6700	0,7900	0,8541
KNN (k=5)	0,8000	0,9500	0,6300	0,7600	0,8197

Fonte: Os autores (2026).

O Random Forest obteve o melhor F1-Score (0,7900) e AUC-ROC (0,8541). Todos os modelos apresentaram alta Precision ($\sim 95\text{--}97\%$), indicando baixa taxa de alarmes falsos, e Recall moderado ($\sim 63\text{--}67\%$), o que significa que aproximadamente um terço dos ataques reais não é detectado.

5.3 Validação Cruzada e OOB Score

A Tabela 2 apresenta os resultados da validação cruzada.

Tabela 2: Resultados da validação cruzada estratificada 5-fold (F1 no treino)

Modelo	F1 Médio	Desvio Padrão	OOB Score
Árvore de Decisão	0,9998	0,0001	N/A
Random Forest	0,9999	0,0001	0,9998
KNN (k=5)	0,9950	0,0010	N/A

Fonte: Os autores (2026).

O OOB Score de 0,9998 confirma a consistência do Random Forest como estimativa independente de generalização. A diferença entre os valores de treino ($\sim 0,9998$) e teste ($\sim 0,79$) é esperada: o NSL-KDD Test+ é propositalmente mais difícil que o conjunto de treino [2].

5.4 Teste de McNemar

O Teste de McNemar retornou $p < 0,05$, confirmando diferença estatisticamente significativa entre Árvore de Decisão e Random Forest. Isso descarta a hipótese de que a diferença de 0,01 em F1-Score se deve ao acaso [5].

5.5 Importância de Features

A Tabela 3 apresenta as cinco features mais importantes identificadas pelo Random Forest. A convergência entre as features da EDA e as do Random Forest valida a coerência do pipeline.

Tabela 3: Top 5 features mais importantes — Random Forest (MDI)

#	Feature	Importância	Interpretação
1	src_bytes	Alta	Ataques exibem padrões bimodais (zero ou muito alto)
2	dst_bytes	Alta	Normais recebem mais dados do destino
3	flag	Alta	Status TCP (SYN, RST) diferencia tipos de tráfego
4	logged_in	Média	Login bem-sucedido indica tráfego legítimo
5	count	Média	Alta contagem em 2s indica varredura ou DDoS

Fonte: Os autores (2026).

Nota: A feature service (66 valores únicos) pode ter importância inflada pelo viés de alta cardinalidade do MDI. Permutation Feature Importance seria uma alternativa mais robusta.

6. DISCUSSÃO

O Random Forest superou os demais modelos em todas as métricas primárias, resultado consistente com a literatura [1, 6]. A combinação de bagging com aleatoriedade nas features produz árvores menos correlacionadas, reduzindo a variância do ensemble e melhorando a generalização — especialmente relevante no NSL-KDD, onde features de escalas muito distintas coexistem.

A alta Precision ($\sim 97\%$) em todos os modelos é vantajosa em ambientes operacionais, pois reduz a fadiga de analistas com alarmes falsos. O Recall moderado ($\sim 65\text{--}67\%$), porém, representa um risco real: em um ambiente com 10.000 tentativas de ataque por dia, cerca de 3.500 passariam despercebidas. Isso reforça a necessidade de camadas adicionais de defesa (análise comportamental, honeypots, correlação de logs).

A Árvore de Decisão, embora com desempenho ligeiramente inferior, é preferível em contextos que exigem auditabilidade das decisões — como ambientes regulados por LGPD ou ISO 27001. O trade-off é quantificado: perde apenas 0,01 em F1 e 0,012 em AUC-ROC para o Random Forest.

A diferença entre os valores de validação cruzada no treino (F1 $\sim 0,9998$) e no teste (F1 $\sim 0,79$) não indica overfitting clássico, mas sim a dificuldade intencional do NSL-KDD Test+, que contém proporções diferentes de tipos de ataque para garantir avaliações mais realistas [2].

7. LIMITAÇÕES E AMEAÇAS À VALIDADE

7.1 Validade Interna

Os hiperparâmetros não foram otimizados via GridSearchCV, o que pode subestimar o desempenho real. O LabelEncoder impõe ordem ordinal nas features categóricas, o que pode introduzir viés em modelos sensíveis a relações de ordem, embora não afete significativamente as árvores de decisão.

7.2 Validade Externa

O NSL-KDD foi gerado em 1999. Ataques modernos — ransomware, APIs REST, IoT, APTs — não estão representados. Os modelos podem apresentar desempenho inferior em tráfego real contemporâneo, onde a distribuição de features e a proporção entre classes de ataque são substancialmente diferentes.

7.3 Ameaças ao Desempenho

O Recall de $\sim 65\%$ é insuficiente para sistemas críticos. Técnicas de balanceamento (SMOTE, *class weighting*) e ajuste do threshold de decisão poderiam melhorar o Recall — a análise da curva Precision-Recall indica que Recall de $\sim 80\%$ é atingível com Precision de $\sim 85\text{--}88\%$.

8. CRONOGRAMA

Tabela 4: Cronograma de desenvolvimento do projeto

Atividade	Fev.	Mar.	Abr.	Mai.	Jun.
Levantamento bibliográfico	x				
Análise exploratória de dados		x			
Pré-processamento e pipeline		x	x		
Treinamento e validação dos modelos			x		
Análise estatística (McNemar)			x		
Interpretação e feature importance				x	
Elaboração do artigo				x	x
Revisão e entrega final					x

Fonte: Os autores (2026).

9. CONCLUSÃO

Este trabalho demonstrou que o Random Forest é o modelo de melhor desempenho para classificação de tráfego de rede no dataset NSL-KDD, com F1-Score de 0,7900, AUC-ROC de 0,8541 e OOB Score de 0,9998, com diferença estatisticamente significativa confirmada pelo Teste de McNemar ($p < 0,05$). A análise de features revelou que `src_bytes`, `dst_bytes` e `flag` são os principais discriminadores entre tráfego normal e malicioso — resultado consistente com os padrões identificados na EDA, o que valida a coerência do pipeline desenvolvido.

O pipeline é completamente reproduzível: sementes aleatórias fixas (`random_state=42`), divisão treino/teste antes do pré-processamento, rastreamento de experimentos em `experiments/experiments`, versões de bibliotecas fixadas em `requirements.txt`, e execução com um único comando (`python main.py`).

Como trabalhos futuros, recomenda-se: (1) ajuste de `threshold` e aplicação de SMOTE para melhorar o Recall; (2) otimização de hiperparâmetros via GridSearchCV; (3) testar redes neurais profundas (MLP, LSTM) para capturar padrões temporais; (4) usar datasets mais recentes como CIC-IDS-2017 ou UNSW-NB15; e (5) implementar Permutation Feature Importance para ranking mais confiável de features, mitigando o viés de cardinalidade do MDI.

REFERÊNCIAS BIBLIOGRÁFICAS

Referências

- [1] TAVALLAEE, M. et al. **A detailed analysis of the KDD CUP 99 data set.** In: *IEEE Symposium on Computational Intelligence for Security and Defense Applications*, 2009. p. 1–6.
- [2] REVATHI, S.; MALATHI, A. **A detailed analysis on NSL-KDD dataset using various machine learning techniques for intrusion detection.** *International Journal of Engineering Research & Technology*, v. 2, n. 12, 2013.
- [3] KAUFMAN, S. et al. **Leakage in data mining: Formulation, detection, and avoidance.** *ACM Transactions on Knowledge Discovery from Data*, v. 6, n. 4, 2012.
- [4] JAPKOWICZ, N.; SHAH, M. **Evaluating Learning Algorithms: A Classification Perspective.** Cambridge: Cambridge University Press, 2011.
- [5] DIETTERICH, T. G. **Approximate statistical tests for comparing supervised classification learning algorithms.** *Neural Computation*, v. 10, n. 7, p. 1895–1923, 1998.
- [6] BREIMAN, L. **Random forests.** *Machine Learning*, v. 45, n. 1, p. 5–32, 2001.
- [7] PEDREGOSA, F. et al. **Scikit-learn: Machine learning in Python.** *Journal of Machine Learning Research*, v. 12, p. 2825–2830, 2011.